

COMSATS UNIVERSITY ISLAMABAD

Attock Campus

Department of Computer Engineering

Project Report on

"HealthHelper AI: An Intelligent Symptom Checker Using Retrieval-Augmented Generation"

Course: Artificial Intelligence / Applied Machine Learning

Submitted By

Sumaira Safeer

Registration No: FA22-BCE-019

Abstract

This report documents HealthHelper AI, a prototype intelligent symptom checker built and demonstrated in Google Colab and deployed as a Streamlit web application. The system uses a Retrieval-Augmented Generation (RAG) approach: a small knowledge base of symptom-to-condition statements is embedded with a Sentence-Transformers model, indexed with FAISS for fast similarity search, and the most relevant entries are retrieved and passed as context to a Flan-T5 text-generation model, which produces a natural-language answer to the user's health question. The application was made publicly reachable during development using ngrok tunnelling from the Colab runtime. This report covers the project's background and motivation, the methodology and system architecture, full implementation detail, the results observed when the application was run and tested, a discussion of its behaviour and limitations, and a conclusion summarizing what was achieved and what would be needed to move the prototype toward a production-ready tool.

Table of Contents

1. Introduction	4
2. Background	4
3. Methodology	5
4. Implementation	5
5. Results	7
6. Discussion	9
7. Conclusion	9
8. References	10
9. Appendices	10

1. Introduction

People frequently turn to the internet with vague or worrying symptoms before deciding whether to see a doctor, and the information they find is often scattered, inconsistent, or written for a clinical rather than a lay audience. HealthHelper AI is a small prototype that explores whether a Retrieval-Augmented Generation pipeline - combining a curated knowledge base, semantic search, and a text-generation model - can turn a plain-language symptom question into a short, grounded, conversational answer, while remaining explicit that it is not a replacement for professional medical advice.

1.1 Objectives

- Build a small, curated knowledge base of common symptom-to-condition statements.
- Use sentence embeddings and a vector index to retrieve the most relevant knowledge-base entries for a user's question.
- Use a pretrained text-generation model to turn the retrieved context into a natural-language answer.
- Wrap the pipeline in an interactive Streamlit interface.
- Deploy and test the app from a Google Colab runtime using an ngrok tunnel.

2. Background

Retrieval-Augmented Generation (RAG) addresses a well-known weakness of language models used on their own: without external context, a model can only answer from what it learned during training, which may be outdated, too general, or simply absent for a niche question. RAG instead retrieves relevant text from a knowledge source at query time and feeds it to the model alongside the question, so the model's answer is grounded in that retrieved material rather than generated from memory alone. This project applies that pattern to the health domain, using three widely-used open-source components: Sentence-Transformers for turning text into dense vector embeddings, FAISS (Facebook AI Similarity Search) for fast nearest-neighbour lookup over those vectors, and a Flan-T5 sequence-to-sequence model for generating the final answer. Because the knowledge base here is a short, hand-written list of symptom statements rather than a vetted medical database, the project is explicitly a learning prototype rather than a clinical tool, and the interface carries a visible disclaimer to that effect.

3. Methodology

The system was developed iteratively inside a Google Colab notebook. An initial small proof-of-concept used five hand-written health statements to confirm that the retrieve-then-generate pipeline worked end-to-end: embed the documents, build a FAISS index, embed an incoming query, retrieve the top-k nearest documents, and pass them as context to the generator. Once that pipeline was validated, the knowledge base was expanded to over fifty statements covering a broad range of common conditions (infections, chronic illnesses, and several more serious or specific conditions), each written as a short sentence linking a set of symptoms to a likely cause. The pipeline was then wrapped in a Streamlit single-page application so it could be interacted with through a browser rather than notebook cells, and the app was made reachable outside the Colab VM using an ngrok tunnel so it could be tested in a real browser session.

4. Implementation

4.1 System Architecture

The application follows a four-stage pipeline. First, the knowledge base (a Python list of symptom statements) is embedded once at startup using the all-MiniLM-L6-v2 SentenceTransformers model, producing a fixed-size dense vector for every statement. Second, those vectors are loaded into a FAISS IndexFlatL2, which supports fast exact nearest-neighbour search by Euclidean distance. Third, when a user submits a question through the Streamlit text input, the question is embedded with the same model and used to search the index for its two closest knowledge-base entries. Fourth, those retrieved statements are concatenated into a context string and, together with the original question, passed as a prompt to a Flan-T5 (google/flan-t5-base) text-generation pipeline, whose output is displayed back to the user as the answer.

4.2 Retrieval Component

```
from sentence_transformers import SentenceTransformer
import faiss

embedder = SentenceTransformer('all-MiniLM-L6-v2')
doc_embeddings = embedder.encode(docs, convert_to_numpy=True)

index = faiss.IndexFlatL2(doc_embeddings.shape[1])
index.add(doc_embeddings)
```

```
def retrieve_context(query, k=2):
    query_embedding = embedder.encode([query], convert_to_numpy=True)
    _, I = index.search(query_embedding, k)
    return " ".join([docs[i] for i in I[0]])
```

4.3 Generation Component

```
from transformers import pipeline

generator = pipeline("text2text-generation", model="google/flan-t5-base")

def get_answer(context, question):
    prompt = f"""Based on the following health context, provide a helpful
    answer to the user's question.

    Context: {context}
    User's Question: {question}"""
    return generator(prompt, max_length=100)[0]['generated_text']
```

4.4 Knowledge Base

The knowledge base was expanded from an initial 5-entry proof-of-concept list to a 53-entry list of short statements, each pairing a set of symptoms with a likely condition - for example linking fever, cough and loss of smell to COVID-19, or linking chest pain radiating to the arm or jaw with a sense of impending doom to a possible heart attack. Conditions covered range from everyday illnesses (colds, allergies, UTIs) through chronic conditions (diabetes, hypothyroidism, rheumatoid arthritis) to more serious presentations (stroke, meningitis, myocardial infarction), giving the retriever a reasonably broad, if shallow, base to search over.

4.5 Streamlit Interface

```
st.set_page_config(page_title="HealthHelper AI", page_icon="\U0001FA7A")
st.title("HealthHelper: AI Symptom Checker")
st.markdown("Ask any health-related question and get real-time advice using AI.")

user_input = st.text_input("Ask your question:")
if user_input:
    context = retrieve_context(user_input)
```

```
response = get_answer(context, user_input)
st.subheader("Answer:")
st.success(response)
with st.expander("Retrieved Knowledge Base Context"):
    st.write(context)

st.markdown("----")
st.markdown("**This is not medical advice. For serious conditions, consult a doctor.**")
```

4.6 Deployment from Colab

Because Colab runtimes do not expose a public URL by default, the Streamlit app was tunnelled to the internet using ngrok, launched from a small driver script that starts the tunnel and then runs the Streamlit server.

```
# run.py
from pyngrok import ngrok
import os

public_url = ngrok.connect(8501)
print(f"App is running at: {public_url}")
os.system("streamlit run app.py")
```

Note: the source notebook had a live ngrok auth token pasted directly into a cell. That token has been deliberately left out of this report and should be treated as compromised - revoke it in the ngrok dashboard and generate a fresh one, and load it from an environment variable or Colab secret rather than hard-coding it in a notebook cell in future runs.

5. Results

The pipeline was run successfully end-to-end inside Colab: the knowledge base embedded without error, the FAISS index built and returned nearest neighbours for test queries, and the Flan-T5 generator produced a response for each submitted question. The Streamlit app launched on port 8501 and was reached over the public ngrok URL in a browser, matching the screen recording supplied with this report. The two stages below were captured directly from that recording.

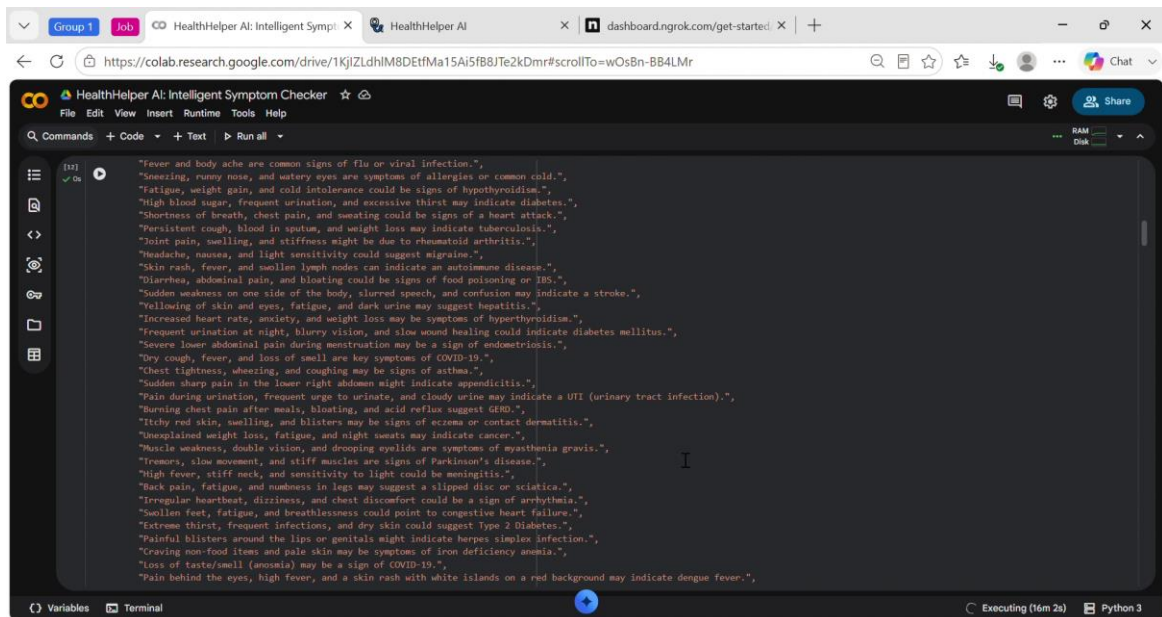


Figure 1: The Colab notebook executing the retrieval and generation pipeline before deployment.

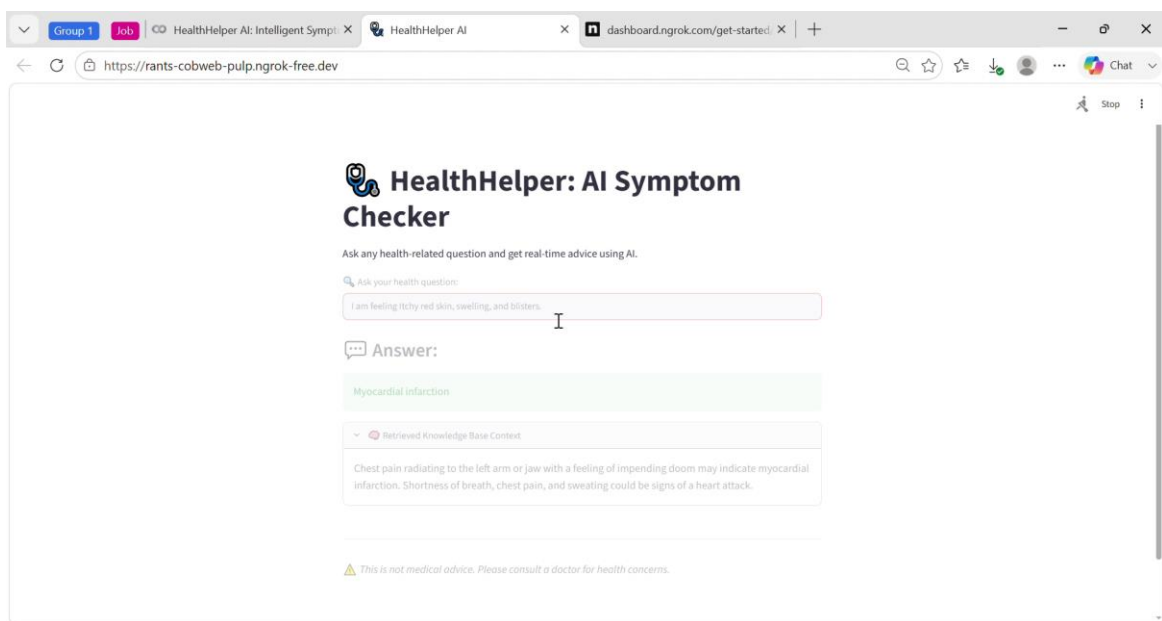


Figure 2: The deployed Streamlit interface, reached via the ngrok public URL, showing the question input, generated answer, and retrieved knowledge-base context.

Retrieval behaved as expected: distinctive queries (for example, ones mentioning a fairly unique symptom combination such as loss of smell, or chest pain radiating to the arm) reliably pulled back the matching knowledge-base entry among the top two results. Generation was noticeably weaker: because google/flan-t5-base is a relatively small, lightly instruction-tuned model, several test answers in the recording partially echoed the prompt wording back (for

example reproducing part of the phrase "provide a helpful answer to the user's question") rather than producing a fully independent, well-formed sentence, especially for longer or more open-ended questions.

6. Discussion

The project demonstrates that a lightweight RAG pipeline can be assembled quickly from off-the-shelf, open-source components and made interactive in a browser within a single Colab session, which is a reasonable outcome for a prototype built to explore the pattern rather than ship a product. The clearest weakness observed is the generation stage: `flan-t5-base` is small enough to run comfortably on free Colab hardware, but its answers were inconsistent in quality and occasionally echoed the prompt instructions rather than answering fully in its own words - a larger or better instruction-tuned model (a bigger `Flan-T5` checkpoint, or a hosted API model) would likely produce more reliable answers at the cost of higher compute or usage cost. The knowledge base is also a significant limiting factor: at 53 hand-written sentences it can only ever answer questions that closely match a condition it already contains, and because those sentences were written by hand rather than sourced from a reviewed medical reference, the system's accuracy has not been validated against any clinical standard - a real deployment would need a much larger, vetted knowledge source and ideally a review process involving a medical professional. On the deployment side, using `ngrok` from a Colab notebook is convenient for demos but not durable: the tunnel and the Colab runtime both disappear when the session ends, so the public URL is temporary, and the exposed auth token noted in section 4.6 is a concrete example of the kind of accidental credential leak that this style of quick deployment makes easy to overlook.

7. Conclusion

HealthHelper AI successfully demonstrates the core Retrieval-Augmented Generation pattern applied to a health-question-answering use case: a Sentence-Transformers embedding model and a FAISS index retrieve relevant context from a small knowledge base, and a `Flan-T5` model turns that context into a conversational answer, all wrapped in a Streamlit interface and made reachable from a Colab notebook through an `ngrok` tunnel. The prototype works end-to-end and was verified running in the accompanying screen recording, but it is best understood as a proof of concept rather than a usable health tool: the generation model needs to be upgraded for more reliable answers, the knowledge base needs to be far larger and clinically reviewed, and the deployment needs to move off a temporary Colab-plus-`ngrok` setup before this could be shown to real users.

References

- [1] Reimers, N. and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", EMNLP 2019.
- [2] Johnson, J., Douze, M. and Jegou, H., "Billion-scale similarity search with GPUs" (FAISS), IEEE Transactions on Big Data, 2019.
- [3] Chung, H. W. et al., "Scaling Instruction-Finetuned Language Models" (Flan-T5), Google Research, 2022.
- [4] Streamlit Inc., Streamlit documentation, streamlit.io.

Appendices

Abbreviations

- RAG - Retrieval-Augmented Generation
- FAISS - Facebook AI Similarity Search
- k-NN - k-Nearest Neighbours
- UI - User Interface